# Этапы подготовки и разработки MVP

*Подробная дорожная карта: от каркаса проекта до проверяемого MVP с multi-camera identity tracking*

Подготовлено на основе совместной архитектурной проработки проекта.
Версия: расширенный черновик v1.

Документ описывает последовательность действий, при которой каждый следующий этап опирается на проверенный предыдущий. Цель — минимизировать хаос, ускорить отладку и дойти до MVP через серию контролируемых вертикальных срезов.

## 1. Принцип выполнения работ

Если бы проект вел senior tech lead, то разработка шла бы не через параллельное написание всех модулей, а через инкременты, каждый из которых дает отдельный проверяемый результат. Такой подход позволяет уже на ранних этапах увидеть реальные ошибки датасета, трекера, crops, identity-логики и не спрятать их под многослойной сложностью.

1. Сначала делается минимальный рабочий runtime.
2. Потом — устойчивый локальный трек.
3. Потом — quality-gated embeddings.
4. Потом — identity внутри одной камеры.
5. Только после этого — multi-camera attach.
6. После attach — face resolution и contradiction flow.
7. И только затем — продуктовая логика и упаковка MVP.

## 2. Ключевые правила управления проектом

• Нельзя открывать новый сложный слой, пока текущий слой не имеет debug artifacts и понятных acceptance criteria.

• Каждый этап обязан заканчиваться демонстрацией: видео, overlay, JSON, crops, граф, метрики.

• Нельзя переходить к multi-camera, пока локальный трек и body crops не стали объяснимыми.

• Каждый этап должен иметь тестовый набор видео и перечень типовых отказов.

• Отладочные инструменты — обязательный deliverable каждого слоя.

## 3. Сводная последовательность этапов

| Этап | Цель | Что получаем | Статус проверки |
|---|---|---|---|
| 0 | Каркас проекта | Структура репозитория, domain models, YAML validation | unit tests + старт сервиса |
| 1 | Video runtime | Ingest + decode + detection + pose | overlay + JSONL |
| 2 | Local tracking | lite/strong трекер, T_lost, reactivation | track replay + ID audit |
| 3 | Geometry | distortion + homography + world point | video + map overlay |
| 4 | Config & primitive events | zones, lines, dwell, line crossing | events vs ground truth |
| 5 | Body pipeline | body crops, quality gate, embeddings, view_type | crop audit |
| 6 | Identity inside one camera | IdentityHypothesis, prototypes, support logic | hypothesis dump |
| 7 | Face & ResolvedPerson | face quality gate, face embeddings, person resolution | retro-link demo |
| 8 | Multi-camera attach | new LocalTrack -> existing IdentityHypothesis | 2–3 camera demo |
| 9 | Contradiction flow | face overrides body, relink, split foundations | corrective scenario |
| 10 | One product MVP use case | end-to-end сценарий для заказчика | demo on target videos |
| 11 | Freeze and stabilize | profiling, cleanup, acceptance checklist | release candidate |

## Этап 0. Подготовка каркаса проекта

Цель этапа: зафиксировать скелет системы и не допустить смешения слоев в будущем

### Что входит в этап

• Создать модульную структуру репозитория: domain, runtime, tracking, identity, config, storage, debug.

• Зафиксировать канонические модели данных: FrameObservation, LocalTrack, BodyObservation, FaceObservation, IdentityHypothesis, ResolvedPerson, PrimitiveEvent.

• Сделать YAML loader и schema validation.

• Определить единый формат логов, debug-артефактов и output directories.
• Подключить минимальный CI для unit tests на модели и конфиги.

### Ожидаемый результат

• Запускаемый каркас приложения без бизнес-логики.
• Валидируемый конфиг и предсказуемый bootstrap runtime.
• Набор dataclass / pydantic моделей, на которых будут строиться остальные модули.

### Как проверить и принять этап

• Сервис стартует на тестовом YAML без падения.
• Unit tests на доменные сущности и конфиг-парсер проходят.
• Логи читаемы и содержат идентификаторы объекта, камеры и модуля.

### Что сознательно не делать на этом этапе

• Не писать здесь трекер, embeddings или граф ассоциации.
• Не зашивать product-specific логику в bootstrap.

### Основные риски

• Если сразу не зафиксировать доменную модель, later-stage коды начнут использовать несовместимые структуры.
• Хаотичный конфиг с первых дней быстро разрушит управляемость проекта.

## Этап 1. Video runtime: ingest + decode + person detection + pose

Цель этапа: получить первый реально работающий видеопайплайн и формат наблюдений

### Что входит в этап

• Подключить видеопотоки из RTSP и/или файлов.
• Поднять decode на выбранном стеке и выдавать кадры в inference pipeline.
• Выполнять person detection + pose на каждом кадре / sampled frames.
• Писать FrameObservation в JSONL и формировать overlay video.

### Ожидаемый результат

• Пайплайн читает поток и стабильно выдает bbox + keypoints + confidence.
• Сформирован канонический выход detection/pose уровня.

### Как проверить и принять этап

• Есть 3–5 тестовых видео с сохраненным overlay.
• JSONL/NDJSON содержит корректные observation records.
• Визуально на видео видно, что люди и позы определяются ожидаемо.

### Что сознательно не делать на этом этапе

• Не добавлять пока local tracking, identity и события.

### Основные риски

• Плохой формат observation на этом шаге потом сломает трекер и quality gates.

• Если не сохранять overlay и JSONL, дальнейшая отладка будет слепой.

## Этап 2. Local tracking v1

Цель этапа: построить устойчивый локальный трек, который станет ядром всей системы

### Что входит в этап

• Реализовать tracking profile lite.
• Реализовать tracking profile strong.
• Ввести состояния new / active / lost / closed.
• Реализовать T_lost и reactivation policy.
• Сохранить историю observation ids внутри LocalTrack.
• Сделать overlay с local_track_id, state, age, lost counter.

### Ожидаемый результат

• Люди внутри одной камеры получают понятный и в целом устойчивый local ID.
• Краткие пропадания не приводят каждый раз к созданию нового трека.

### Как проверить и принять этап

• Нужно минимум 3 класса видео: одиночный человек, несколько людей, окклюзии.
• Сделать ручной audit ID switches и false new tracks.
• Сделать replay одного трека по timeline: create → active → lost → reactivate/close.

### Что сознательно не делать на этом этапе

• Не тянуть сюда multi-camera continuity.
• Не использовать лицо или глобальный ID для исправления локального трекера.

### Основные риски

• Слабый local tracking убьет весь верхний стек независимо от качества embeddings.
• Если strong profile не будет заметно сильнее lite, продуктовое разделение профилей потеряет смысл.

## Этап 3. Геометрия как optional слой

Цель этапа: подключить distortion correction и homography, не ломая базовый runtime

### Что входит в этап

• Реализовать загрузку calibration artifacts.
• Поддержать distortion correction.
• Реализовать вычисление world point и запись WorldState.
• Отображать tracks на floor plan и сохранять confidence world projection.

### Ожидаемый результат

• Для камер с калибровкой есть точка на плане и trajectory overlay.
• При отсутствии калибровки пайплайн продолжает работать без геометрии.

### Как проверить и принять этап

• Подготовить 1–2 калиброванные камеры и эталонные видео.
• Сравнить траекторию в image-space и floor-map.
• Проверить, что отключение geometry не ломает runtime.

### Что сознательно не делать на этом этапе

• Не включать геометрию в identity gating v1.
• Не пытаться на этом этапе строить сложные карты переходов между камерами.

### Основные риски

• Опорная точка может улетать при окклюзии; нужен quality-aware update.
• Попытка сделать geometry mandatory резко снизит переносимость коробки.

## Этап 4. YAML-конфиг и primitive events

Цель этапа: перенести управление базовой логикой из кода в конфиг

### Что входит в этап

• Описать камеры, sampling, tracking profile, geometry, zones и lines в YAML.
• Поддержать line crossing, enter/exit zone, dwell time.
• Писать PrimitiveEvent в хранилище или JSONL.
• Сделать валидацию rule templates и params.

### Ожидаемый результат

• Простая аналитика управляется конфигом без изменения кода.
• Есть события, которые можно сверять вручную с видео.

### Как проверить и принять этап

• Подготовить конфиг для 1 камеры с несколькими зонами и линией.
• Сверить события вручную на тестовом видео.
• Проверить invalid config и читаемость ошибок валидации.

### Что сознательно не делать на этом этапе

• Не пытаться пока сделать универсальный UI-конструктор.
• Не добавлять произвольный пользовательский скриптовый язык.

## Этап 5. Body pipeline: crops, quality gate, embeddings

Цель этапа: получить качественные body samples как основу будущего identity

### Что входит в этап

• Определить crop policy для тела.
• Реализовать quality gate по bbox size, area, occlusion, pose validity, stability.
• Определять view_type: front/back/left/right.
• Извлекать embeddings только для валидных samples.
• Сохранять body crops и metadata в audit folders.

### Ожидаемый результат

• Для каждого LocalTrack появляется набор качественных BodyObservation.
• Мусорные и маленькие crops перестают попадать в identity-слой.

### Как проверить и принять этап

• Ручной аудит body crops обязателен: папки по track_id, внутри crops и JSON с quality fields.
• Отдельно проверить front/back/left/right cases.
• Проверить, что embeddings не пишутся на откровенно плохих samples.

### Что сознательно не делать на этом этапе

• Не строить сразу multi-camera attach по одному embedding.
• Не усреднять все body embeddings человека в один вектор.

### Основные риски

• Если quality gate слабый, vector DB быстро заполнится мусором.
• Ошибки в view_type резко ухудшат retrieval по hypotheses.

## Этап 6. IdentityHypothesis внутри одной камеры

Цель этапа: доказать работоспособность middle identity layer до multi-camera

### Что входит в этап

• Реализовать сущность IdentityHypothesis и BodyPrototype.
• Создавать hypothesis после N_create_support_min качественных observations.
• Хранить prototypes по bucket'ам front/back/left/right.
• Связывать LocalTrack с hypothesis и обновлять support_count / confidence.
• Сделать dump hypotheses в JSON.

### Ожидаемый результат

• Система умеет строить body identity без лица хотя бы в границах одной камеры.
• Можно отдельно проверить логику создания и обновления hypothesis.

### Как проверить и принять этап

• Взять видео с повторными появлениями человека и контролировать, как создаются hypotheses.
• Проверить, что hypotheses не плодятся из-за одного слабого sample.
• Проверить ограничение K_max_per_view.

### Что сознательно не делать на этом этапе

• Не начинать еще межкамерную continuity.
• Не создавать ResolvedPerson без лица.

## Этап 7. Face pipeline и ResolvedPerson

Цель этапа: добавить верхний уровень подтверждения личности

### Что входит в этап

• Поднять face detection и face quality gate.

• Извлекать FaceObservation и face embeddings.
• Поднять face index / lookup.
• Реализовать создание или привязку ResolvedPerson.
• Сделать retro-link history: hypothesis и связанные tracks наследуют person link.

### Ожидаемый результат

• Сценарий 'сначала hypothesis, потом confirmed person' работает end-to-end.
• Появление лица не ломает историю body identity, а уточняет ее.

### Как проверить и принять этап

• Подготовить видео, где лицо видно только в конце маршрута.
• Проверить корректность retro-link history и audit trail.
• Проверить сценарий без match и с созданием нового person, если такая логика разрешена.

### Что сознательно не делать на этом этапе

• Не использовать лицо как костыль для исправления слабого локального трекера.
• Не пытаться в этом этапе реализовать полнофункциональный person search UI.

## Этап 8. Multi-camera body association v1

Цель этапа: получить первую рабочую continuity между камерами через identity hypotheses

### Что входит в этап

• Поднять ANN/KNN retrieval по body prototypes hypotheses.
• Использовать attach policy с retrieve / provisional / confirm thresholds и candidate_margin_min.
• Поддержать association states: candidate_found, provisional_link, confirmed_link, contradicted.
• Привязывать новый LocalTrack к существующей hypothesis только после нескольких подтверждений.
• Сделать визуальный и JSON debug для межкамерной привязки.

### Ожидаемый результат

• На 2–3 камерах один человек получает continuity через общую hypothesis.
• При этом local_track_id на каждой камере остается локальным и независимым.

### Как проверить и принять этап

• Собрать тестовый набор: один человек между камерами, похожие люди, униформа, разные ракурсы.
• Считать false attaches и missed attaches вручную или через простые метрики.
• Проверить, что ambiguous cases не склеиваются агрессивно.

### Что сознательно не делать на этом этапе

• Не добавлять temporal/camera/world gating в первой итерации.
• Не подтверждать attach по одному body sample.

### Основные риски

• Без gating возрастет риск ложных merge — поэтому пороги и margin должны быть консервативными.

• Если hypotheses уже были зашумлены на предыдущих этапах, multi-camera attach станет нестабильным.

## Этап 9. Contradiction flow и foundations для split/merge

Цель этапа: сделать систему способной исправлять ошибки, а не только накапливать их

### Что входит в этап

• Реализовать contradiction state.
• Если лицо противоречит body attach, помечать старую связь как contradicted.
• Выполнять relink track/hypothesis/person.
• Сохранять audit trail причин и результатов исправления.
• Заложить структуры под будущие split/merge operations.

### Ожидаемый результат

• Система умеет не только связывать identity, но и корректировать ошибочные связи.
• Есть воспроизводимый сценарий исправления body-ошибки по лицу.

### Как проверить и принять этап

• Смоделировать ложный attach и позднее появление качественного лица.
• Проверить, что relink не стирает историю, а переопределяет верхний уровень ссылок.
• Проверить дамп графа после contradiction.

## Этап 10. Один end-to-end MVP use case

Цель этапа: собрать вертикальный продуктовый сценарий, который можно показывать заказчику

### Что входит в этап

• Выбрать один главный кейс: например, waiting without staff в КСО или поиск человека между камерами.
• Привязать его к YAML-конфигу, events и identity layer.
• Сделать понятный output: events JSON, debug video, explanation trail.

### Ожидаемый результат

• Рабочий демонстрационный сценарий на реальных видео.
• Сценарий показывает ценность платформы, а не только внутреннюю архитектуру.

### Как проверить и принять этап

• Сверка результата с expected business logic.
• Демонстрация для внутренней команды без ручных пост-фиксов.
• Проверка, что pipeline воспроизводим на одном и том же конфиге.

## Этап 11. Freeze MVP, profiling и стабилизация

Цель этапа: остановить рост scope и довести систему до состояния рабочего MVP

### Что входит в этап

• Профилировать latency и использование ресурсов.
• Очистить временный код и хаки.
• Собрать acceptance checklist по всем предыдущим этапам.
• Зафиксировать known issues и backlog v2.
• Подготовить пакет демо-материалов и короткую эксплуатационную документацию.

### Ожидаемый результат

• Release candidate, который можно разворачивать на пилоте и обсуждать с заказчиком.
• Понятный список ограничений v1 и roadmap следующей итерации.

### Как проверить и принять этап

• Regression прогон на эталонных видео.
• Повторная проверка YAML сценариев.
• Ручной просмотр debug overlays и дампов identity-графа.

### Что сознательно не делать на этом этапе

• Не добавлять в этот момент новые фичи типа 3D pose или complex graph heuristics.

## 4. Артефакты, которые должны появляться по мере этапов

• Overlay videos по каждому ключевому слою: detection/pose, local tracking, identity attach.
• JSONL/NDJSON с observations, tracks и events.
• Папки с body crops и face crops.
• Dumps identity hypotheses и full graph snapshots.
• Список test videos и known failure cases.
• Acceptance checklist на каждый этап.

## 5. Что должно быть готово до начала multi-camera этапа

1. Локальный трекер стабилен и объясним.
2. Body crops проходят ручной аудит и не зашумлены мусором.
3. View type определяется достаточно надежно для first-pass retrieval.
4. IdentityHypothesis уже работает хотя бы внутри одной камеры.
5. Есть отладочные дампы hypotheses и tracks.

## 6. Минимальная граница MVP

MVP не должен пытаться охватить все блоки платформы. Достаточно следующего состава:

• 2–3 камеры;
• person detection + pose;
• local tracking;
• body observations и hypotheses;
• basic multi-camera continuity;
• face confirmation;

• один законченный продуктовый сценарий;
• YAML config;
• debug tooling.

Все, что выходит за этот набор, является post-MVP развитием и не должно тормозить первую итерацию.

## 7. Рекомендуемый формат управления спринтами

Практически проект удобно вести короткими спринтами по 1–2 недели на этап, но с обязательной внутренней демонстрацией в конце. На демо показывается не код, а артефакты: видео, overlays, crops, графы, events, конфиг.

| Параметр | Описание |
|---|---|
| Формат спринта | Задачи на один вертикальный слой + обязательный debug pack. |
| Обязательный выход спринта | Demo video, набор логов/JSON, краткий changelog, список известных проблем. |
| Критерий перехода дальше | Можно руками проверить, что текущий слой работает и понятен. |
| Красный флаг | Следующий этап начинается, хотя по текущему слою нет overlays/crops/graph dumps или команда 'примерно понимает', как он работает. |

## 8. Резюме для команды разработки

Если держать эту последовательность, проект будет развиваться как серия контролируемых шагов: каждый слой сначала доказывает свою жизнеспособность отдельно, после чего становится основой следующего слоя. Это сильно снижает риск утонуть в сложной мультикамерной логике до того, как будут доказаны базовые предпосылки: хороший local tracking, хорошие body crops и корректная hypothesis model.